

PERSONAL EXPENSE TRACKER

A File-Based Python Project

VITyarthi – Build Your Own Project
Project Report

Student Details	To be filled before submission
Student Name	
Registration Number	
Course / Subject	
Faculty Name	

Technology: Python 3 • Storage: CSV + JSON • Interface: Command Line

1. Introduction

The Personal Expense Tracker is a console-based application designed to help an individual record, review and analyse day-to-day expenses. The project focuses on applying programming concepts such as modular design, classes, functions, file handling, input validation, collections, sorting and basic aggregation in a practical problem.

The application deliberately does not use a database. Expense records are persisted in a CSV file and monthly budgets are persisted in a JSON file. This makes the project portable, easy to install, easy to understand and suitable for a beginner-level academic demonstration. The application can run without MySQL or any third-party Python package.

The VITyarthi instructions require an original project relevant to the course, at least three major functional modules, a clear input/output structure and a logical workflow. They also require documentation, design artefacts, modular implementation, validation/error handling, testing and Git usage. The present design is structured around those requirements. ■filecite■turn0file0■L5-L20■

2. Problem Statement

Students frequently make small payments for food, transportation, education, shopping, entertainment and other needs. When these transactions are not recorded consistently, it becomes difficult to know how much has been spent during a month or which categories consume the largest portion of the available budget.

The problem addressed by this project is therefore: **How can an individual record expenses in a simple offline application and obtain a useful monthly summary without depending on a database server?**

3. Objectives

- Provide a simple interface for entering daily expenses.
- Store expense records permanently between program runs.
- Allow incorrect expense records to be deleted.
- Allow a user to set a monthly spending budget.
- Generate monthly total and category-wise spending summaries.
- Validate user input before saving data.
- Demonstrate modular Python programming and file handling.
- Provide a project that can be explained and demonstrated during evaluation.

4. Scope

Included: expense creation, expense listing, deletion, monthly budgets, monthly reports, category aggregation, validation, local persistence and automated validation tests.

Not included: online banking integration, payment processing, cloud synchronisation, multi-user authentication, database management, mobile application support and machine-learning prediction. These are intentionally outside the current scope so that the implementation remains focused and explainable.

5. Functional Requirements

The VITyarthi specification asks for at least three major functional modules and a logical input/output workflow.

■filecite■turn0file0■L24-L37■

ID	Module / Requirement	Input	Output
FR-01	Expense Management	Date, category, amount, description	Saved expense / validation errors
FR-02	Expense Viewing & Deletion	Menu choice, optional expense ID	Expense list / deletion confirmation

ID	Module / Requirement	Input	Output
FR-03	Budget Management	Month and budget amount	Saved monthly budget
FR-04	Reporting & Analytics	Month	Total, category totals, budget and remaining amount
FR-05	Validation	User-entered values	Valid data or meaningful error messages

6. Non-Functional Requirements

The project defines more than the required four non-functional requirements. The VITyarthi document lists qualities such as performance, security, usability, reliability, scalability, maintainability, error handling and resource efficiency.

filecite turn0file0 L38-L50

Requirement	Project Implementation
Usability	Numbered menu, clear prompts and readable reports.
Performance	Small CSV/JSON files are processed quickly for normal personal use.
Reliability	Data is stored locally and recreated with headers/files if missing.
Maintainability	Separate modules for models, validation, storage, expenses, budgets and reports.
Error Handling	Invalid dates, amounts, categories, descriptions and IDs are rejected with messages.
Resource Efficiency	Only Python standard-library modules are used; no database server is required.
Scalability	The architecture can later be extended with a GUI or database without changing every module.
Security	No passwords or financial account credentials are collected; data remains on the local machine.

7. System Architecture

The application follows a simple layered architecture. The user interacts with the console interface in **main.py**. The interface calls service modules for expenses, budgets and reports. These modules use the storage layer, which reads/writes CSV and JSON files.

```

User
↓
Console Interface (main.py)
↓
+-----+-----+-----+
| ExpenseManager | BudgetManager | ReportService |
+-----+-----+-----+
↓
Storage Layer
/ \
↓ ↓
expenses.csv budget.json

```

This structure keeps user-interface logic separate from business logic and persistence. The technical expectations specifically ask for proper architecture, modular/clean implementation, documentation, validation and testing where applicable. filecite turn0file0 L51-L63

8. Design Diagrams

8.1 Use Case Diagram

```

+-----+
| PERSONAL EXPENSE TRACKER |
+-----+
User -----| Add Expense |
User -----| View Expenses |
User -----| Delete Expense |
User -----| Set Monthly Budget |
User -----| Generate Monthly Report |
+-----+

```

8.2 Workflow Diagram

```

START → Main Menu → Select Operation
↓
+-----+-----+
| | |
Expense Budget Report
| | |
Validate Validate Read Data
| | |
Save Save Calculate
| | |
+-----+-----+
↓
Main Menu → EXIT

```

8.3 Sequence Diagram – Add Expense

```

User → main.py: enter date/category/amount/description
main.py → validators.py: validate input
validators.py → main.py: valid OR errors
main.py → ExpenseManager: add()
ExpenseManager → storage.py: save_expenses()
storage.py → CSV file: write record
ExpenseManager → main.py: new expense ID
main.py → User: success message

```

8.4 Component/Class Diagram

```

Expense { expense_id, date, category, amount, description }
Budget { month, amount }

ExpenseManager
+ add()
+ delete()
+ list_all()

BudgetManager
+ set_budget()
+ get_budget()

ReportService
+ monthly_summary()
+ dashboard()

Storage
+ load_expenses()
+ save_expenses()
+ load_budget()
+ save_budget()

```

8.5 File Storage Design

File	Purpose	Fields / Structure
expenses.csv	Persistent expense records	expense_id, date, category, amount, description
budget.json	Persistent monthly budgets	JSON object: month → amount

9. Design Decisions & Rationale

- No database: the user explicitly requested a project without a database, and local files are sufficient for a single-user academic application.
- CSV for expenses: human-readable, easy to inspect and edit, and naturally represents tabular records.
- JSON for budgets: monthly budgets are naturally represented as key-value pairs.
- Standard library only: reduces installation problems and makes the project easy to run on a lab computer.
- Modular structure: each responsibility is placed in a separate file, making debugging and future changes easier.
- Command-line interface: keeps the implementation focused on core Python concepts rather than GUI complexity.

10. Implementation Details

10.1 models.py

Defines the Expense and Budget data classes. Data classes provide a clear representation of the two main types of information used by the program.

10.2 validators.py

Checks positive numeric amounts, valid YYYY-MM-DD dates, allowed categories and non-empty descriptions. Validation occurs before an expense is saved.

10.3 storage.py

Creates the data directory/files when needed, reads CSV records into Expense objects, writes Expense objects back to CSV, and reads/writes monthly budgets as JSON.

10.4 expense_manager.py

Implements the expense-management operations: add, delete and list. IDs are generated sequentially from the current maximum ID.

10.5 budget_manager.py

Validates the YYYY-MM month format and positive budget amount, then saves or retrieves a monthly budget.

10.6 reports.py

Filters expenses by month, calculates the total, groups spending by category and compares the result with the configured monthly budget.

10.7 main.py

Provides the menu-driven user interface and connects user input to the application services.

11. Program Flow and Algorithms

11.1 Add Expense Algorithm

- Read date, category, amount and description.
- Validate all values.
- Load existing expenses from CSV.
- Find the next available ID.
- Create an Expense object.
- Append it to the list.
- Save the complete list back to CSV.
- Display the generated ID.

11.2 Monthly Report Algorithm

- Read the requested month.
- Load all expenses.
- Select records whose date begins with the requested YYYY-MM value.
- Group selected amounts by category.

- Calculate the overall total.
- Read the monthly budget, if one exists.
- Calculate remaining budget = budget – total.
- Display the results.

12. User Interface / Sample Results

```

=== PERSONAL EXPENSE TRACKER ===
1. Add expense
2. View expenses
3. Delete expense
4. Set monthly budget
5. Monthly report
6. Exit
Choose an option: 1
Date [2026-09-26]: 2026-09-26
Categories: Food, Travel, Education, Shopping, Entertainment, Bills, Health, Other
Category: Food
Amount (■): 150
Description: Lunch
Expense added successfully. ID: 4
REPORT – 2026-09
Total spent: ■700.00
Budget: ■5,000.00
Remaining: ■4,300.00

Category-wise spending:
Education ■350.00
Food ■270.00
Travel ■80.00

```

The example above demonstrates the intended input/output flow. During the final demonstration, the student should run the program and capture actual screenshots from their own computer for this section.

13. Testing Approach

Testing focuses on validation because incorrect user input is the most likely failure point in a small console application. Unit tests are included for positive/negative amounts, correct/incorrect date formats and accepted/rejected categories. Python syntax compilation was also checked for all project modules.

Test ID	Test Case	Expected Result	Status
TC-01	Amount = 250.50	Accepted	PASS
TC-02	Amount = -1	Rejected	PASS
TC-03	Date = 2026-09-26	Accepted	PASS
TC-04	Date = 26-09-2026	Rejected	PASS
TC-05	Category = Food	Accepted	PASS
TC-06	Category = Unknown	Rejected	PASS

Automated test execution result: **3 unit-test methods passed**. Python bytecode compilation also completed without syntax errors.

14. Challenges Faced

- Designing persistent storage without relying on a database.
- Keeping CSV data consistent while adding and deleting records.
- Validating user input before it reaches the storage layer.

- Keeping the project modular enough to satisfy academic documentation requirements while remaining easy to explain.
- Generating useful monthly summaries from simple file-based records.

15. Learnings & Key Takeaways

- Learned how to divide a real-world problem into functional modules.
- Practised Python classes and data classes.
- Applied file handling with CSV and JSON.
- Understood the importance of input validation and error handling.
- Used lists, dictionaries, sorting and aggregation to process records.
- Learned why separation of responsibilities improves maintainability.
- Practised writing unit tests for reusable validation functions.
- Understood how a project can be documented using requirements, architecture, workflow and UML-style diagrams.

16. Future Enhancements

- Add a Tkinter graphical user interface.
- Add charts for category-wise and monthly spending.
- Add an expense-edit feature.
- Add recurring-expense support.
- Add export to PDF/Excel.
- Add search and date-range filters.
- Add optional password protection for the local files.
- If future project requirements allow it, replace the storage layer with a database without changing the main business modules.

17. GitHub Repository Structure

The VITYarhi instructions require a GitHub repository containing README.md, statement.md and complete organized source files. The README is expected to describe the title, overview, features, technologies, installation/run steps and testing instructions. ■filecite■turn0file0■L87-L106■

```
Expense_Tracker_NoDB/
■ ■ ■ main.py
■ ■ ■ models.py
■ ■ ■ config.py
■ ■ ■ validators.py
■ ■ ■ storage.py
■ ■ ■ expense_manager.py
■ ■ ■ budget_manager.py
■ ■ ■ reports.py
■ ■ ■ utils.py
■ ■ ■ requirements.txt
■ ■ ■ README.md
■ ■ ■ statement.md
■ ■ ■ data/
■ ■ ■ expenses.csv
■ ■ ■ budget.json
■ ■ ■ tests/
■ ■ ■ test_project.py
■ ■ ■ docs/
■ ■ ■ diagrams.md
```

18. Step-by-Step Installation and Execution

Step 1: Install Python 3.10+ on the computer. During Windows installation, enable “Add Python to PATH”.

Step 2: Extract the project ZIP into a convenient folder such as Desktop.

Step 3: Open the extracted **Expense_Tracker_NoDB** folder.

Step 4: Open Command Prompt or PowerShell in that folder.

Step 5: Check Python using **python --version**. If that command does not work on Windows, try **py --version**.

Step 6: Start the application using **python main.py** or **py main.py**.

Step 7: Select options 1–6 from the menu and enter the requested values.

Step 8: To test the project, run **python -m unittest discover -s tests -v**.

Step 9: For GitHub, initialise Git, add the files, commit them and push the repository to GitHub. No database setup is required.

19. GitHub Submission Commands

```
git init
git add .
git commit -m "Initial Expense Tracker project"
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

20. Conclusion

The Personal Expense Tracker provides a complete small-scale software solution for recording and analysing personal expenses. It demonstrates a clear problem statement, functional modules, non-functional requirements, modular Python implementation, file-based persistence, validation, reporting and testing. The design is intentionally database-free so that the project can be executed on a normal computer with Python alone. The project therefore remains simple enough for a student to understand while still providing sufficient technical depth for a structured academic submission.

21. References

- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines, supplied as the project specification.
- Python Standard Library documentation concepts: csv, json, pathlib, dataclasses, datetime and unittest.

The VITyarthi source specifies the report sections including cover page, introduction, problem statement, requirements, architecture, diagrams, design rationale, implementation, results, testing, challenges, learnings, future enhancements and references. ■filecite■turn0file0■L109-L134■